

Computation and Boundary Evidence for Scientific Software Repair

Rajarshi Ghoshal

rajarshi.ghoshal1@gmail.com

PhD applicant research task — Graz University of Technology

Abstract

A repair agent with shell tools must reconstruct a repository’s computational structure from text search. CBE (computation and boundary evidence) prepares that structure once and lets the agent query it during repair, under the same model and budget. On 89 held-out SWE-bench Science tasks ($k=3$), both arms solve 60 attempts (22.9% vs. 23.1%). Aggregate parity masks a discipline-level split: mechanics +22.2 pp, materials +9.7 pp, astronomy +12.4 pp; biomedical engineering −16.7 pp, biology −9.5 pp. CBE generates 13.7% fewer output tokens. About a fifth of tasks flip between identical runs, bounding what any single-run comparison can resolve.

1 Introduction

Repairing a composite-laminate stiffness calculation requires changing both the ply iteration and the accumulation formula. Expanding one without the other passes the public check but fails the private tests. The patch is short; finding it means knowing which expression produces the returned matrix and what calls cross the repository boundary.

We give the agent that knowledge as a prepared store and test whether it changes outcomes. CBE exposes source-linked computation structure; the baseline reconstructs the same information from `grep` and file reads. Same model, same budget.

Contributions: the preparation pipeline, the query interface, and a replicated evaluation that separates discipline-level differences from aggregate noise.

CBE draws on traceability (Niu et al., 2023b,a, 2025; Pan et al., 2025), code property graphs (Yamaguchi et al., 2014), scientific model extraction (Pyarelal et al., 2020; ML4AI Team, 2024), and incremental parsing (Brunsfeld et al., 2024). We evaluate on SWE-bench Science (Xu et al., 2026) with DeepSeek V4.1 Flash (DeepSeek, 2026); SWE-bench (Jimenez et al., 2024; Yang et al., 2024) and SciCode (Tian et al., 2024) are related benchmarks.

Table 1. Evidence fields, from task 103’s saved records.

Record	Content
Computation	Scope, expressions, result anchors: accumulation at line 1003 → <code>return ABD</code> at 1015.
Condition	<code>len(z0) == len(z1)</code> at line 987.
Definition	Documented public behavior (not execution trace).
Boundary	<code>np.radians(thetad)</code> at line 979: provider <code>numpy</code> , caller-side scope.
Query interface (alongside ordinary shell)	
<code>find</code>	Targets and source excerpts.
<code>inspect</code>	Relationships, source, definitions, or contracts.
<code>note</code>	Optional working model; never gates a repair.

2 What CBE provides

Prepare. Run the public `reproduce.py` under an observer. Observed functions guide file selection. Static extraction adds expressions, conditions, definitions, and call boundaries. No LLM call. Median duration: 73 s of the 1800 s budget.

Represent. Tree-sitter (Brunsfeld et al., 2024) parses Python. Joern handles C, C++, Fortran, and Cython where available; explicit fallback otherwise. External calls are labelled with their provider and marked as interface assumptions, not correctness guarantees. Unresolved callees stay `unknown_external`.

Query and repair. The agent gets the three endpoints from Table 1 plus its normal tools. Inspection refreshes edited text; the preparation trace is not re-executed on candidates.

Table 2. Shared conditions.

Component	Detail
Model	DeepSeek V4.1 Flash (DeepSeek, 2026); temperature 0; PYTHONHASHSEED=0.
Replicates	89 tasks $\times$ 3 runs = 267 scheduled per arm; 522 verified.
Budget	1800 s, including preparation.
Verification	Separate container, 1800 s, runs after repair ends. Private tests not visible.
Network	None.
Baseline	Shell tools + public task material.
CBE	Same + evidence store + three query endpoints.

Table 3. Both arms solve 60. CBE generates 13.7% fewer output tokens.

Metric	Baseline	CBE
Scheduled attempts	267	267
Exact repairs / verdicts	60/262	60/260
Exact-repair rate	22.9%	23.1%
Private tests passed	2315/3167	2226/3148
Output tokens (M)	12.01	10.36
Input tokens (M)	838.0	878.3
Mean work per attempt (s)	1236.6	1306.9

Paired task-rate diff: +0.75 pp; 95% bootstrap $[-3.4, +4.9]$ pp; sign $p = 1.00$.

3 Protocol

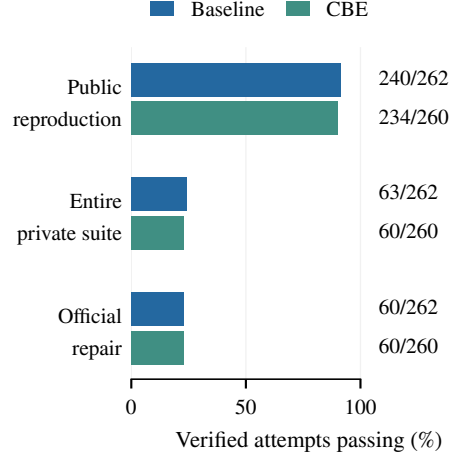
119 tasks, 98 repositories, 20 disciplines (Xu et al., 2026). 30 for development, 89 locked for evaluation.

A solve is the official verifier reward. Missing verdicts excluded, not failures. Paired test: bootstrap per-task rate differences (20k resamples, fixed seed). Sign test excludes ties.

4 Parity, fewer tokens

74 tasks tie, 7 favour CBE, 8 favour baseline. Output drops 13.7% (10.36M vs. 12.01M tokens). Input rises 4.8%. Work takes 70 s longer. The agent searches less — it either finds the right computation or anchors on the wrong one.

Hidden partial changes. Of 74 exact ties, 30 differ in private-test coverage (Table 4).

**Figure 1.** Most attempts pass the public check but not the full verifier.**Table 4.** Exact-rate ties crossed with private-test coverage.

Exact rate	Private-test coverage		
	Base >	Equal	CBE >
Baseline higher	7	0	1
Equal	21	44	9
CBE higher	0	0	7

5 Discipline split

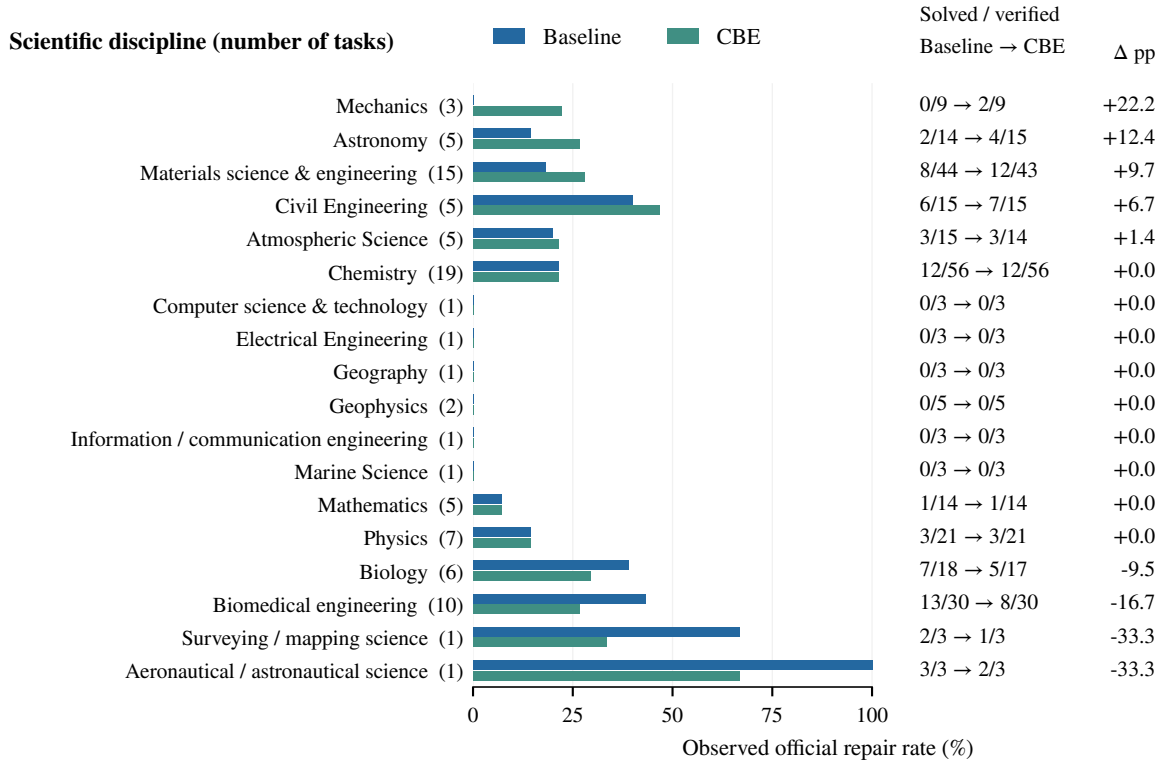


Figure 2. Parity hides opposing domain effects. All 18 represented disciplines; task counts in parentheses; right column is the percentage-point change.

Mechanics: +22.2 pp, entirely task 103 (pyNas-tran). The agent inspects the ply-angle computation and its numpy boundaries, then edits — 2/3 solves vs. 0 baseline. Materials: +9.7 pp across three tasks. Astronomy: +12.4 pp.

Biomedical engineering: −16.7 pp across four tasks. Task 022 (nilearn): baseline solves 2/3, CBE 0/3. The trajectory reads projection mathematics before touching the atlas-label mapping. Biology: −9.5 pp. The two −33.3 pp domains are singletons.

Chemistry (19 tasks), physics (7) and mathematics (5) do not move at all. The ablation flag shows no interaction: −0.3 pp on ablated tasks, +1.4 pp on knowledge-provided ones. Language strata are uninformative, since 81 of the 89 tasks are pure Python.

When the defect is a computation, structure finds it. When an ordinary bug sits under scientific prose, structure distracts.

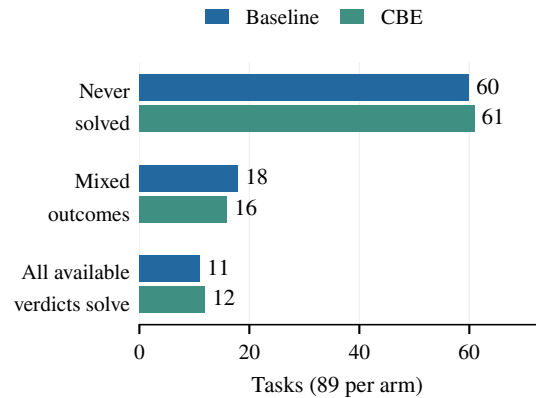


Figure 3. Most tasks never solved or always solved. About a fifth flip within an arm.

6 Case studies

Task / repository	Discipline	Baseline			CBE			Solved / verified
		Run 1	Run 2	Run 3	Run 1	Run 2	Run 3	
074 DeePTB	Materials	9/10	9/10	2/10	10/10	10/10	—	0/3 → 2/2
042 sunpy	Astronomy	15/16	15/16	6/16	16/16	7/16	16/16	0/3 → 2/3
103 pyNastran	Mechanics	2/4	2/4	2/4	4/4	4/4	2/4	0/3 → 2/3
067 xclim	Atmospheric	6/6	6/6	3/6	6/6	6/6	—	2/3 → 2/2
020 deepmd-kit	Materials	10/12	10/12	11/12	10/12	12/12	10/12	0/3 → 1/3
084 dpdata	Materials	5/7	5/7	7/7	7/7	5/7	7/7	1/3 → 2/3
116 Pynite	Civil eng.	85/93	85/93	85/93	85/93	93/93	85/93	0/3 → 1/3
029 biopython	Biology	29/29	14/29	27/29	14/29	14/29	14/29	1/3 → 0/3
053 MONAI	Biomedical	2/6	6/6	2/6	5/6	5/6	2/6	1/3 → 0/3
063 tskit	Biology	9/18	18/18	7/18	7/18	9/18	—	1/3 → 0/2
087 dipy	Biomedical	5/5	0/5	1/5	0/5	2/5	2/5	1/3 → 0/3
093 MONAI	Biomedical	4/4	2/4	2/4	2/4	2/4	2/4	1/3 → 0/3
098 satpy	Surveying	11/11	8/11	11/11	8/11	11/11	8/11	2/3 → 1/3
056 poliastro	Aeronautical	10/10	10/10	10/10	10/10	10/10	1/10	3/3 → 2/3
022 nilearn	Biomedical	12/12	12/12	10/12	4/12	11/12	11/12	2/3 → 0/3

Cell: private tests passed / collected. Outlined + bold: official repair. Dash: no verdict.

Figure 4. Per-run private-test fractions for every task whose solve rate differs between arms. Outlined bold cells are official solves; bold task labels are the two cases below.

Table 5. What changed in the patches.

Task	Baseline patches	CBE patches	What it shows
103 pyNastran	Expand ply iteration, keep original B/D accumulation. All three pass the public check.	Two change both material-reference expansion and B/D expressions; the third keeps the old accumulation.	The public check undertests. Covering the full computation, not just the example, separates the two complete patches from the three partial ones.
022 nilearn	Two add a label-aggregation option without altering the existing nearest-neighbour path.	Two add label-aware aggregation but also change how the existing path is selected or computed; 11/12 private tests pass.	Adding a capability while preserving the existing interface is the obligation. CBE’s trajectory reads projection maths instead of the interface contract.

Task 103 shows that covering the documented computation, not just the example discrepancy, is the repair obligation. Task 022 shows that preserving the existing interface while adding the requested case is. CBE helped with the first and hurt the second.

7 Limitations and conclusion

One model, one effort, one budget. Domain analyses are descriptive; singleton domains carry no weight. The store is approximate: unparseable expressions are recorded as unsupported, external calls are labelled rather than verified. No ablations isolate which part of CBE matters.

CBE reaches parity, generates fewer tokens, and splits the benchmark by discipline. Scientific structure helps when the defect is a computation and

hurts when it is ordinary code.

Artifacts and AI use. The author designed the study; LLM assistants (Oh My Pi, ChatGPT) helped implement the harness and revise prose. DeepSeek V4.1 Flash (DeepSeek, 2026) was the only repair model.

References

- Max Brunsfeld et al. Tree-sitter: An incremental parsing system for programming tools, 2024. URL <https://tree-sitter.github.io>.
- DeepSeek. DeepSeek V4.1 Flash: Model card and API documentation, 2026. URL <https://api-docs.deepseek.com>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- ML4AI Team. GroMEt: A unified intermediate representation for scientific models and code. *DARPA ASKEM / SKEMA Consortium*, 2024.
- Feifei Niu, Wesley K. G. Assunção, Liguó Huang, Christoph Mayr-Dorn, Jidong Ge, Bin Luo, and Alexander Egyed. RAT: A refactoring-aware traceability model for bug localization. In *Proc. 45th International Conference on Software Engineering (ICSE)*, 2023a.
- Feifei Niu, Christoph Mayr-Dorn, Wesley K. G. Assunção, Liguó Huang, Jidong Ge, Bin Luo, and Alexander Egyed. The ABLoTS approach for bug localization: Is it replicable and generalizable? In *Proc. 20th International Conference on Mining Software Repositories (MSR)*, pages 152–162, 2023b.
- Feifei Niu, Liguó Huang, Christoph Mayr-Dorn, and Alexander Egyed. TVR: Automotive system requirement traceability validation and recovery through retrieval-augmented generation. *arXiv preprint arXiv:2504.15427*, 2025.
- Xinyi Pan, Feifei Niu, Liguó Huang, and Alexander Egyed. Requirements coverage-guided minimization for natural language test cases. *arXiv preprint arXiv:2505.20004*, 2025.
- Adarsh Pyarelal et al. AutoMATES: Automated model assembly from text, equations, and software. *arXiv preprint arXiv:2001.07295*, 2020.
- Minyang Tian, Luyu Gao, Shizhuo Dylan Zhang, et al. SciCode: A research coding benchmark curated by scientists. *arXiv preprint arXiv:2407.13168*, 2024.
- Zhipeng Xu, Jiahao Lu, Yining Zheng, Yuxin Wang, and Xipeng Qiu. SWE-bench Science: Can coding agents resolve engineering tasks in science? *arXiv preprint arXiv:2608.19799v2*, 2026.
- Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. Modeling and discovering vulnerabilities with code property graphs. In *IEEE Symposium on Security and Privacy*, 2014. doi: 10.1109/SP.2014.44.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-bench Verified: A stricter subset for real-world software engineering. *arXiv preprint arXiv:2407.01489*, 2024.